Information Security Lab (HDU IS Labs)

Work 1 — Secure REST API Development & CVE Reproduction

Name: Zhang Jiexiang
Student ID: 23320317

1. Screenshots

1.1 Shellshock Reproduction vs. Remediation Comparison

Figure 1 is one terminal window with two requests, one above the other. The top one is victim.cgi with the malicious User-Agent, and it printed back the output of the command I injected, uid=33(www-data). The bottom one is the same header sent to safe.cgi, and the page came back normal.

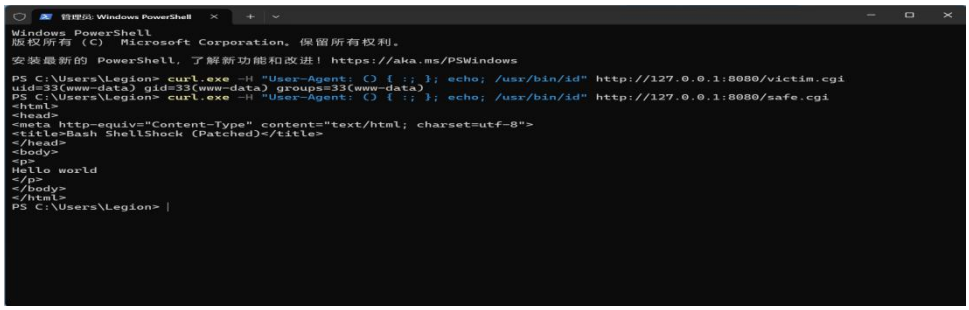


Figure 1 Shellshock reproduction vs. remediation (victim.cgi vs. safe.cgi)

1.2 API Testing and CI Pipeline Verification

Besides the Shellshock part, I also had to show that the API itself worked and that the CI pipeline passed:

- API: I logged in, put the token in the Authorization header, and GET /api/data returned 200 with the protected records. Without a token the same call returned 401.
- JWT check: three requests with curl.exe. Log in and get a token, call the protected endpoint with it, then call it with no token at all. All three as expected.
- CI: bandit for static analysis and OWASP Dependency-Check both ran inside GitHub Actions and both passed. No critical dependency was reported and the workflow finished green.

2. Difference Analysis

2.1 Behavior Comparison of victim.cgi and safe.cgi

I sent the same payload to both scripts, with User-Agent: () { :; }; echo; /usr/bin/id. They behaved very differently:

	Unpatched bash 4.3.0 (/usr/local/bash-4.3.0/bin/bash)	**Patched bash (/bin/bash)**	**Core difference: Bash version**
Response status	HTTP 200, body is uid=33(www-data) gid=33(www-data) groups=33(www-data)	HTTP 200, normal HTML page (Hello world)	Command was injected and executed on the vulnerable version
Command output	Yes (result of /usr/bin/id)	No	The patch successfully blocked command execution

So it comes down to the bash version. The old bash reads an environment variable starting with () { as a function definition and does not stop at the closing brace, so my echo and id still ran. The patched bash stops at the end of the body and drops what follows.

2.2 Root Cause Analysis of CVE-2014-6271

CVE-2014-6271, Shellshock, hits unpatched bash from 1.14 to 4.3. I split the cause into two parts:

(1) The bug in bash. When bash imports an environment variable starting with () {, it treats it as a function definition but does not stop at the closing brace. It keeps parsing and runs what comes after the semicolon, and that is the injection.

(2) The way the payload reaches bash. Apache turns request headers into environment variables, so my User-Agent arrived as HTTP_USER_AGENT and was handed to the interpreter of the CGI script. I never touched the server directly, I only wrote a header, and bash ran my command. That is remote code execution.

2.3 Remediation Principles

What I actually changed in the lab, and the wider advice it maps to:

- Upgrade bash. Anything at 4.3-29 or newer, or 4.4 and above, has the fix. The patch changes the parser for imported functions, so this is a root cause fix, not a filter.
- Do not feed user data into the environment. Headers are user input, and user input should not end up in the environment of a bash process.
- Make the CGI side smaller. Pass fewer headers through, or run the scripts with an interpreter that is not bash.
- Put a WAF or IDS in front. It cannot repair the bug, but blocking requests that contain () { stops most attempts.

In my setup safe.cgi only points its shebang at /bin/bash, the patched one on that image. That single line was enough to stop the payload.

3. Personal Contributions

I did this lab on my own. The parts I am responsible for:

- The API. I wrote the FastAPI service from an empty folder: JWT login with PyJWT (HS256), bcrypt for the password hashes, parameterized queries so the SQL cannot be injected, and three routes, login, data and profile.
- Repository and CI. I created the GitHub repository, wrote the Actions workflow and put bandit and Dependency-Check inside it. Every run I did finished green.
- The CVE part. I started the vulhub image (vulhub/bash:4.3.0-with-httpd), sent the malicious User-Agent until a command ran, then wrote safe.cgi as the fixed version and repeated the same request against it. The comparison is in Figure 1, and the longer write-up is in fix/CVE-2014-6271-Shellshock-Remediation-Report.md.

4. Lessons Learned and Pitfalls

Three things went wrong before the lab worked. They cost me more time than the code itself:

- PyJWT wants sub to be a string. I passed the user id as an integer and got a validation error while the token was being created. Wrapping it as str(user_id) fixed it.
- bandit was too strict for me. It raised low severity warnings on ordinary code such as logging and temp file writes, and those still failed the job. I added --exit-zero so warnings show up without breaking the run.

- The worst one was the shebang. I edited victim.cgi on Windows, saved it with CRLF, and every request came back 500. The shebang line ended with a hidden \r, so the kernel looked for an interpreter with a carriage return in its name and gave up. Converting to LF fixed it. I did not know line endings alone could break a CGI script.

Running the same request twice, once against the vulnerable image and once against the patched one, is what made this lab click for me. SQL injection and XSS used to be bullet points in the OWASP list. This was also the first time I had a scanner running inside CI instead of by hand at the end.